Reverse

Base64

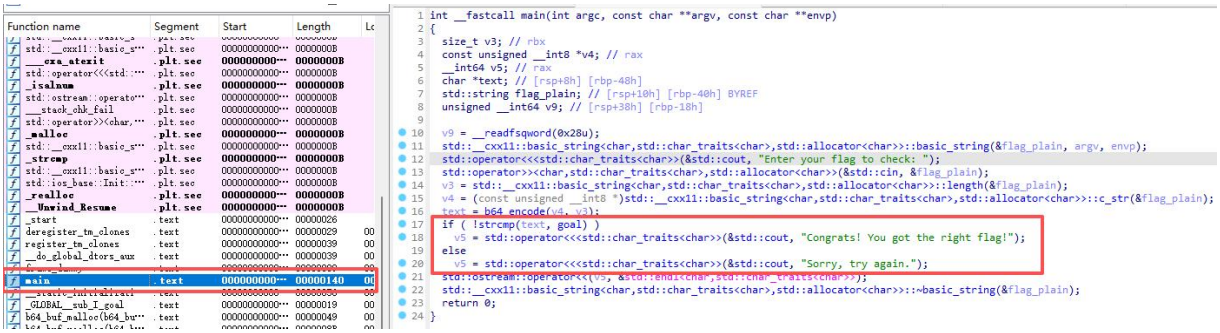
本题在某年的 mini-L 中出现过类似的签到题。

拿到题目后解压，发现一个无后缀的文件，下意识的在 linux 上看下，目的是确认文件类型、符号、是否被 UPX 打包、是否有直接明文 flag。

```
(base) gaile@gaile-virtual-machine:~/ctf$ file flag1
flag1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically link
ed, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=c3e494b362e69e05707b1
d8650afba8a958968f5, for GNU/Linux 3.2.0, with debug_info, not stripped
```

直接查看文件头时发现：这是一个 ELF 可执行文件（不是纯文本的 base64 文件）。题目是 base64，但上传的是一个可执行程序，那么 flag 很可能内嵌在二进制里或者由二进制在运行时解密输出。

于是将文件放入 IDA 中，找到 main 函数，F5 反编译一下看看里面的代码：



程序把用户输入的明文进行 Base64 编码（b64_encode），然后把编码结果与程序内的 goal 字符串做 strcmp。因此要得到 flag，只要把二进制里存着的 goal 这个 Base64 字符串解码一次即可。

通常情况下 goal 存在.rodata 字符串区，利用 IDA 查看该区域，发现了若干长的 base64-like 字符串

```
.rodata:0000000000002020 ; const char b64_table[66]
.rodata:0000000000002020 _ZL9b64_table   db '18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz'
.rodata:0000000000002061                 db 0
.rodata:0000000000002062                 align 8
.rodata:0000000000002068 aTl0ptwxbkwnvna db 'TL0PTWxbkwNvNaRsyMeP6gNrkwRvwWRPkL0vwKNP6vqbT/qa3Q9BTK/YsDzz',0
.rodata:0000000000002068                                         ; DATA XREF: .data:goal↑o
.rodata:00000000000020A5 aEnterYourFlagT db 'Enter your flag to check: ',0
.rodata:00000000000020A5                                         ; DATA XREF: main+28↑o
.rodata:00000000000020C0 aCongratsYouGot db 'Congrats! You got the right flag!',0
.rodata:00000000000020C0                                         ; DATA XREF: main+9B↑o
.rodata:00000000000020E2 aSorryTryAgain  db 'Sorry, try again.',0
.rodata:00000000000020E2                                         ; DATA XREF: main:loc_13D1↑o
.rodata:00000000000020F4                 align 20h
.rodata:0000000000002100 ; const char b64_table[66]
.rodata:0000000000002100 _ZL9b64_table_0 db '18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz'
.rodata:0000000000002141                 db 0
.rodata:0000000000002142                 align 20h
.rodata:0000000000002160 ; const char b64_table[66]
.rodata:0000000000002160 _ZL9b64_table_1 db '18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz'
.rodata:0000000000002160                                         ; DATA XREF: b64_decode_ex+137↑o
.rodata:0000000000002160                                         ; b64_decode_ex+285↑o
.rodata:00000000000021A1                 db 0
.rodata:00000000000021A2                 align 20h
.rodata:00000000000021C0 ; const char b64_table[66]
.rodata:00000000000021C0 _ZL9b64_table_2 db '18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz'
.rodata:00000000000021C0                                         ; DATA XREF: b64_encode+112↑o
.rodata:00000000000021C0                                         ; b64_encode+210↑o
.rodata:0000000000002201                 db 0
.rodata:0000000000002201 _rodata         ends
.rodata:0000000000002201
```

我尝试解码上面获得到的字符串，理论上就会输出类似 flag{...}的字符串，但却得到

了乱码，不是直接的明文 flag。所以我怀疑 b64_encode 使用自定义 Base64 编码。

于是我返回 IDA 中，在.rodata 区域找到程序使用的自定义 base64 字母表_ZL9b64_table：18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz，于是写了个 python 脚本实现自定义 Base64 解码器：

```python
import base64

# 自定义 Base64 字符表 (64 个字符 + 1 个填充字符 '=')
CUSTOM_ALPHABET = '18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz'
PADDING_CHAR = '='

# 目标编码字符串
goal_variable = 'TL0PTWxbkwNvNaRsyMeP6gNrkwRvwWRPkL0vwKNP6vqbT/qa3Q9BTK/YsDzz'

def custom_b64_decode(encoded_string, alphabet, padding_char):

    # 1. 创建解码映射表
    DECODE_MAP = {char: i for i, char in enumerate(alphabet)}

    # 2. 预处理：移除填充字符
    encoded_string = encoded_string.rstrip(padding_char)

    # 3. 检查长度：Base64 编码后的长度必须是 4 的倍数
    if len(encoded_string) % 4 == 1:
        raise ValueError("Invalid Base64 string length")

    # 4. 解码过程
    decoded_bytes = bytearray()

    for i in range(0, len(encoded_string), 4):
        chunk = encoded_string[i:i + 4]

        b64_values = []
```

```
        for char in chunk:
            if char in DECODE_MAP:
                b64_values.append(DECODE_MAP[char])
            else:
                raise ValueError(f"Invalid character '{char}' in encoded string")

        data_block = 0
        for val in b64_values:
            data_block = (data_block << 6) | val

        num_b64_chars = len(b64_values)

        if num_b64_chars >= 2:
            # 第一个字节 (Bit 23-16)
            decoded_bytes.append((data_block >> 16) & 0xFF)

        if num_b64_chars >= 3:
            # 第二个字节 (Bit 15-8)
            decoded_bytes.append((data_block >> 8) & 0xFF)

        if num_b64_chars >= 4:
            # 第三个字节 (Bit 7-0)
            decoded_bytes.append(data_block & 0xFF)

    return bytes(decoded_bytes)

try:
    decoded_bytes = custom_b64_decode(goal_variable, CUSTOM_ALPHABET, PADDING_CHAR)

    flag = decoded_bytes.decode('utf-8')

    print(f"自定义 Base64 字符表: {CUSTOM_ALPHABET}")
    print(f"原始编码: {goal_variable}")
    print("-" * 30)
    print(f"解码后的字节: {decoded_bytes}")
    print(f"解析得到的 Flag: {flag}")

except ValueError as e:
    print(f"解码错误: {e}")
```

最后得到 flag{base64_translate_table_can_be_changed}

```
自定义 Base64 字符表: 18AtF9QM=ehJpNiZDRoIG/EwkT364yfs+PbaYvLg7uldrxBVC0njX2KWS5mUHqOcz
原始编码: TL0PTWxbkwNvNaRsyMeP6gNrkwRvwWRPkL0vwKNP6vqbT/qa3Q9BTK/YsDzz
------------------------------
解码后的字节: b'flag{base64_translate_table_can_be_changed}\x10@'
解析得到的 Flag: flag{base64_translate_table_can_be_changed}⍰@
```

经验证，是正确的 flag。

```
(base) gaile@gaile-virtual-machine:~/ctf$ ./flag1
Enter your flag to check: flag{base64_translate_table_can_be_changed}
Congrats! You got the right flag!
```